SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME:Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

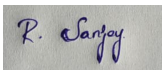
4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I Sanjay Raghavendra/ 192411041 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

ct associated with specific dataset characteristics. Basis-path testing examines internal control flow, while structured prime/composite and partition-boundary tests address the observed failure. Combining these methods improves mathematica

ANSWERS :

1. Online Banking Fund Transfer – Equivalence Partitioning, Boundary Value Analysis and Random Testing

Introduction

An online banking system frequently fails during fund transfers, particularly at the minimum and maximum transfer limits. These failures indicate that the existing test cases do not adequately cover boundary conditions and invalid input values. Since banking applications process financial transactions, testing must ensure that transfers are accurate, secure, and reliable.

Analysis of the Existing Testing Strategy

The testing team may be concentrating on normal transfer amounts while neglecting values at the boundaries of the permitted range. Random testing alone cannot guarantee that important boundary values are tested. Consequently, defects in input validation, transaction processing, and balance updating may remain undetected.

Application of Equivalence Class Partitioning (ECP)

Equivalence Class Partitioning divides input data into groups that are expected to behave similarly. Assuming the permitted transfer amount is between a minimum limit L and maximum limit U, the following classes can be tested:

1. Amount below the minimum limit – invalid.
2. Amount equal to the minimum limit – valid, if the limit is inclusive.
3. Amount between the minimum and maximum limits – valid.
4. Amount equal to the maximum limit – valid, if the limit is inclusive.
5. Amount above the maximum limit – invalid.

6. Zero or negative transfer amount – invalid.
7. Blank or non-numeric amount – invalid.

A representative value from each class should be tested.

Application of Boundary Value Analysis (BVA)

Boundary Value Analysis focuses on values at and around the limits because defects frequently occur at these points.

For the minimum limit L, test $L - 1$, L, and $L + 1$. For the maximum limit U, test $U - 1$, U, and $U + 1$, using the smallest supported currency unit.

For each test, verify the transaction result, account balances, error messages, and transaction history.

Evaluation of Random Testing

Random testing can discover unexpected inputs and unusual failures. However, it may miss important boundary values and does not guarantee systematic coverage. Therefore, random testing should be retained as a supplementary technique rather than the primary testing method.

Proposed Improved Testing Strategy

The improved approach should combine ECP, BVA, risk-based testing, automated regression testing, and random testing. Additional scenarios should include insufficient balance, invalid beneficiary details, daily transfer limits, network failure, and repeated transaction submission.

Conclusion

ECP systematically covers valid and invalid input classes, while BVA targets defects around transfer limits. Combining these techniques with supplementary random testing provides more effective defect detection and improves the reliability of online banking transactions.

2. Healthcare Application – White-Box Testing and Control Flow Graph

Introduction

A healthcare application developed defects after deployment because the testing process did not provide sufficient coverage. Healthcare software must process patient information accurately and reliably. Inadequate testing of decision conditions and execution paths can result in incorrect records, failed updates, and unauthorized access.

Analysis of the Existing Testing Approach

The development team may have relied mainly on statement coverage. Statement coverage checks whether executable statements have been executed at least once. However, it does not guarantee that both outcomes of every decision have been tested.

For example, a patient-validation statement may execute for a valid record, while the invalid-record branch remains untested.

Construction of a Conceptual Control Flow Graph

A conceptual Control Flow Graph (CFG) for the healthcare application can be represented as follows:

Start → Validate Patient Record → Is the Record Valid?

- If Yes, check whether all required clinical fields are complete.
- If the fields are complete, process and save the patient record.
- If the fields are incomplete, display a validation error and stop processing.
- If the patient record is invalid, display an error and stop processing.
- If saving fails, activate the appropriate error-handling procedure.

The graph contains decision nodes, processing nodes, and paths leading to successful or unsuccessful outcomes.

Evaluation of Coverage Criteria

Statement coverage: Ensures that each executable statement runs at least once. It may miss untested decision

outcomes.

Branch coverage: Ensures that every decision outcome, such as True and False, is executed at least once.

Path coverage: Examines sequences of branches through the program. Complete path coverage may be impractical when loops or numerous conditions exist.

Improved Testing Strategy

The testing team should construct the CFG from the actual source code and identify all important decision points. Test cases should cover valid patient records, invalid records, missing fields, unauthorized access, database failures, and unsuccessful save operations.

The team should aim for complete statement and branch coverage of the relevant module, use basis-path testing for independent paths, and apply risk-based testing to critical healthcare functions. Every test should be linked to a requirement, and regression tests should be executed after defects are corrected.

Conclusion

White-box testing helps identify untested branches and internal execution paths. Combining CFG analysis, branch coverage, basis-path testing, and risk-based validation improves healthcare application reliability and reduces the possibility of critical defects reaching production.

3. Retail E-Commerce System – ECP, BVA and Decision Table Testing

Introduction

A retail e-commerce system calculates order totals using price, quantity, and discount. Errors may occur when these inputs interact, even when each input works correctly in isolation. For example, an excessive discount may produce an incorrect total, or a quantity boundary may cause an incorrect calculation.

Analysis of the Existing Approach

Testing each input independently is insufficient because the final calculation depends on combinations of inputs. A test suite containing only ordinary prices, quantities, and discounts may fail to detect invalid combinations, rounding errors, and incorrect business rules.

Application of Equivalence Class Partitioning

The input fields can be divided into valid and invalid classes.

Price: Positive valid price, zero price, negative price, blank price, and non-numeric price.

Quantity: Valid positive integer quantity, zero quantity, negative quantity, and invalid fractional quantity where only whole units are permitted.

Discount: Valid discount within the permitted range, negative discount, discount above the permitted maximum, and malformed discount input.

Each class should contain representative test values.

Application of Boundary Value Analysis

BVA should be applied to the minimum and maximum permitted price, quantity, and discount.

For example, if the permitted discount range is 0% to 100%, test −1%, 0%, 1%, 99%, 100%, and 101%, where these values match the system's input precision.

The actual limits must be taken from the application requirements.

Application of Decision Table Testing

Decision Table Testing helps identify combinations of conditions and their corresponding results. Relevant scenarios include:

1. Valid price, valid quantity, and zero discount.
2. Valid price, valid quantity, and permitted discount.
3. Valid inputs with the maximum permitted discount.
4. Invalid price with otherwise valid inputs.
5. Invalid quantity with otherwise valid inputs.

6. Discount exceeding the permitted limit.
7. Combinations involving bulk quantities and applicable discounts.

Where the number of combinations is large, pairwise testing can supplement the suite, while high-risk business rules must still be tested explicitly.

Improved Testing Strategy

For every test case, verify the calculated subtotal, discount amount, applicable tax, final total, validation messages, and order-submission behavior. Compare the output against an independently calculated expected result.

Conclusion

ECP reduces unnecessary repetition by grouping similar inputs. BVA detects defects near numerical limits, and decision tables identify errors caused by combinations of conditions. Together, these techniques provide more complete and efficient testing of the e-commerce calculation process.

4. Software Module with Nested Conditions and Loops – Path Testing

Introduction

A software module containing nested conditions and loops may experience defects even when the testing team reports partial path coverage. These defects may arise because some branches, loop exits, or independent execution paths have not been tested.

Analysis of the Existing Approach

Partial path coverage indicates that the test suite does not exercise all identified paths. The control flow graph may be incomplete, or the team may have overlooked False branches, loop termination conditions, and exception-handling paths.

Executing a statement once does not establish that all relevant program paths work correctly.

Construction of the Control Flow Graph

The team should create a CFG containing:

- Entry and exit nodes.
- Processing statements.
- Decision nodes with True and False edges.
- Loop-entry and loop-exit edges.
- Error-handling and exception paths.

This graph provides a structured representation of the program's possible execution flow.

Cyclomatic Complexity

Cyclomatic complexity estimates the number of linearly independent paths in a program.

It can be calculated using:

\[
V(G)=E-N+2P
\]

Where:

- $V(G)$ = Cyclomatic complexity.
- E = Number of edges.
- N = Number of nodes.
- P = Number of connected components.

For a single connected program graph, $P = 1$, so:

\[
V(G)=E-N+2
\]

The calculated value helps determine the minimum number of independent basis paths required for basis-path testing.

Proposed Path Testing Strategy

The testing team should identify independent paths and create test cases to execute each path. Both outcomes of every decision should be tested.

For loops, test zero iterations where possible, one iteration, two iterations, typical iterations, and the maximum permitted number of iterations. Nested loops should be tested with empty inner collections, single-item collections, and multiple-item collections.

The team should also examine break and continue statements, invalid inputs, and exception-handling paths.

Complete path coverage may be impossible when loops allow an unlimited number of executions. Therefore, the team should use basis-path coverage, loop-boundary testing, and risk-based testing, while documenting infeasible paths.

Conclusion

CFG construction and cyclomatic complexity provide a systematic foundation for path testing. Testing independent paths and loop boundaries improves branch coverage and helps detect defects that may remain hidden during ordinary execution.

5. Financial Application – Hybrid Black-Box and White-Box Testing

Introduction

A financial application may continue to produce production defects even when both black-box and white-box testing are performed. This suggests that the testing methods may not adequately cover business requirements, internal execution paths, boundary conditions, or error-handling logic.

Analysis of the Existing Strategy

Black-box testing verifies application behavior without examining internal implementation details. White-box testing examines internal logic, branches, and execution paths.

If these techniques are used independently without common requirements and coverage goals, important defects may remain undetected. Coverage percentages alone also cannot prove that the application is correct.

Black-Box Testing Strategy

Apply Equivalence Class Partitioning to inputs such as transaction amounts, account details, account status, and authorization information.

Apply Boundary Value Analysis to minimum and maximum transaction amounts, account-balance thresholds, daily transfer limits, and date-related rules.

Decision Table Testing should cover combinations such as:

- Sufficient balance and valid beneficiary.
- Insufficient balance and valid beneficiary.
- Valid balance and invalid beneficiary.
- Active account and authorized transaction.
- Blocked account or unauthorized transaction.

Each combination should produce the expected transaction result.

White-Box Testing Strategy

Construct a CFG for the transaction-processing module, including input validation, authorization, balance checking, debit, credit, transaction logging, and error recovery.

Test both outcomes of every decision and identify independent basis paths. Include exception paths and rollback behavior.

For example, if a credit operation fails after a debit operation begins, the system should follow the specified recovery process and avoid leaving the accounts in an inconsistent state. Repeated requests should not create

duplicate transactions.

Integration of Both Approaches

Create a requirements traceability matrix connecting requirements, test cases, program paths, and identified defects. Measure statement and branch coverage, apply basis-path testing, and automate regression tests.

Testing effort should be prioritized according to financial risk. Money-transfer and balance-update functions require stronger evidence than low-risk display features.

Conclusion

A hybrid testing strategy combines external behavioral validation with internal control-flow analysis. Integrating ECP, BVA, CFG analysis, path testing, and regression testing improves coverage and reduces the likelihood of important financial defects reaching production.

6. HR Payroll System – Bonus Calculation Using BVA and ECP

Introduction

An HR payroll system calculates monthly bonuses based on employees' years of service and performance ratings. Incorrect payouts for employees with exactly 5, 10, and 15 years of service indicate possible boundary-related defects.

Analysis of the Failure

The problem may result from incorrect comparison operators or off-by-one errors. For example, a condition using greater-than instead of greater-than-or-equal-to may assign an employee to the wrong bonus category.

The existing testing approach may have tested ordinary service durations but failed to examine exact threshold values.

Application of Boundary Value Analysis

Assuming bonus categories change at 5, 10, and 15 years, test values immediately below, exactly at, and immediately above each threshold.

If service is recorded in whole years, use:

- 4, 5, and 6 years.
- 9, 10, and 11 years.
- 14, 15, and 16 years.

If service is calculated using months or days, test the corresponding values immediately before and after each threshold, including the exact threshold date.

The actual bonus rules must determine whether each boundary belongs to the lower or higher category.

Application of Equivalence Class Partitioning

Divide employees into service-duration classes, such as below 5 years, 5 to below 10 years, 10 to below 15 years, and 15 years or more, subject to the specified payroll rules.

Performance ratings should also be divided into valid categories, such as low, medium, and high ratings if these categories are defined by the system. Invalid or missing ratings should be tested separately.

Test representative combinations of service classes and performance ratings.

Improved Testing Strategy

Verify the bonus amount, selected bonus rate, rounding behavior, payslip details, and total payroll calculation for every test case. Automate the boundary tests and execute them whenever the payroll calculation logic changes.

Exploratory testing may help discover unexpected defects, but it does not systematically guarantee coverage of known thresholds.

Conclusion

BVA directly targets the service-duration boundaries where incorrect payouts have occurred. ECP ensures that each service and rating category is represented. These techniques provide more systematic and

repeatable coverage than exploratory testing alone.

7. Laboratory Temperature Control System – CFG and MC/DC Testing

Introduction

An automated laboratory temperature controller uses conditional logic to operate heaters, coolers, and alarms. A failure to trigger an alarm during sensor malfunction indicates that important decision combinations or error-handling paths were not adequately tested.

Since the controller operates in a laboratory environment, incorrect behavior may affect equipment, experiments, or safety.

Construction of the Control Flow Graph

A conceptual CFG can be described as follows:

Start → Read Temperature and Sensor Status → Is the Sensor Valid?

- If the sensor is invalid, activate the alarm and enter the specified safe state.
- If the sensor is valid, compare the temperature with the permitted thresholds.
- If the temperature is below the lower threshold, activate the heater.
- If the temperature is above the upper threshold, activate the cooler.
- If the temperature is within the permitted range, keep the heater and cooler off.
- Update the system status and continue monitoring.

Additional paths should represent sensor-read failures, out-of-range readings, alarm recovery, and hardware failures.

Evaluation of Branch Coverage

Branch coverage requires every decision outcome to be executed. However, a decision can have both True and False outcomes tested without demonstrating that each individual condition independently affects the result.

Therefore, branch coverage alone may be insufficient for complex safety-critical Boolean expressions.

Application of Modified Condition/Decision Coverage (MC/DC)

MC/DC requires each atomic condition in a decision to be shown to independently affect the overall decision outcome.

For example, consider:

\[
D=A\lor B
\]

To demonstrate that condition A independently affects the result, compare:

- $A = True, B = False$, giving $D = True$.
- $A = False, B = False$, giving $D = False$.

Only A changes, and the overall decision changes.

To demonstrate the independent effect of B, compare:

- $A = False, B = True$, giving $D = True$.
- $A = False, B = False$, giving $D = False$.

For the actual controller, the same principle must be applied to its real Boolean expressions.

Improved Testing Strategy

The team should measure statement, branch, and MC/DC coverage for the relevant safety-critical logic. Test sensor malfunction, extreme temperatures, disconnected sensors, and alarm recovery using a simulator or hardware-in-the-loop environment.

Conclusion

MC/DC provides stronger evidence than statement or branch coverage alone for complex Boolean decisions.

Combining CFG analysis, MC/DC, fault-injection testing, and hardware validation improves the reliability of the laboratory temperature controller.

8. Travel Booking Portal – Decision Table Testing for Promo Codes

Introduction

A travel booking portal allows users to apply promotional codes based on booking price, promotion type, and user tier. Certain combinations result in a negative total amount, indicating that the system may not correctly validate combined discounts.

Analysis of the Existing Approach

Testing each input separately cannot reveal all interaction defects. For example, two promotional codes may each be valid independently but invalid when applied together. Similarly, a user may receive a promotion for which their subscription tier is not eligible.

Application of Decision Table Testing

Decision Table Testing maps combinations of conditions to expected system actions.

The relevant conditions include:

- Whether the booking price is valid.
- Whether a promotional code is valid.
- Whether the promotion is fixed-value or percentage-based.
- Whether the user's tier is eligible.
- Whether multiple codes are submitted.
- Whether the combined discount exceeds the booking price.

Expected actions should include accepting the booking, rejecting an invalid promotion, rejecting an ineligible code, applying permitted discounts, or rejecting an invalid combination according to the business requirements.

Proposed Optimized Test Cases

1. Valid booking price with a valid promotion and eligible user tier.
2. Valid booking price without a promotional code.
3. Zero or invalid booking price.
4. Expired or invalid promotional code.
5. Valid promotion applied by an ineligible user.
6. Two compatible promotional codes, if stacking is permitted.
7. Two incompatible promotional codes.
8. Discount exactly equal to the booking price.
9. Combined discount greater than the booking price.

Repeat high-risk cases for fixed-value and percentage-based discounts and the applicable user tiers.

Improved Testing Strategy

For every test case, verify the discount calculation, order in which discounts are applied, rounding, final payable amount, and booking acceptance or rejection.

The system must follow the specified business rules and must not silently generate a negative payable amount.

Use decision-table coverage for meaningful business rules and supplement it with boundary tests around price and discount limits. Pairwise testing can help cover remaining interactions without repeating unnecessary test cases.

Conclusion

Decision Table Testing identifies errors caused by interactions among price, promotional codes, and user tiers. Combining it with boundary and pairwise testing improves defect detection while keeping the test suite

efficient.

9. Database Migration Tool – Infinite Loops and Basis-Path Testing

Introduction

A database migration tool uses nested loops to validate records. Although the testing team reports high statement coverage, infinite-loop crashes continue to occur in production.

This indicates that executing individual statements was not sufficient to establish correct loop termination and control flow.

Analysis of Statement Coverage

Statement coverage verifies that executable statements have run at least once. It does not guarantee that loop exit conditions, zero-iteration cases, or all nested-loop combinations have been tested.

Infinite loops may result from incorrect termination conditions, counters that are not updated, or error-handling paths that return to the wrong location.

Construction of the Control Flow Graph

Construct a CFG that represents record iteration, field validation, nested child-record processing, error handling, and loop exits.

Identify decision nodes and loop back edges. Ensure that all valid and invalid execution paths are represented.

Cyclomatic Complexity and Basis-Path Testing

Calculate cyclomatic complexity using:

\[
V(G)=E-N+2P
\]

For a single connected graph:

\[
V(G)=E-N+2
\]

The result indicates the number of linearly independent basis paths needed for basis-path testing.

The team should identify these paths and design tests that execute each one. Cyclomatic complexity does not represent the total number of possible execution paths.

Loop Boundary Testing Strategy

Test the following situations:

- Zero records.
- One record.
- Two records.
- A normal batch of records.
- The maximum permitted batch size.
- Records containing invalid fields.
- Records with empty child collections.
- Records with one or multiple child items.
- Records that trigger early termination or error handling.

Verify that loop variables are updated correctly and that every loop has a reachable termination condition. Apply execution timeouts to detect non-termination.

Compare the source and destination record counts and confirm that valid records are migrated correctly while invalid records are handled according to the requirements.

Conclusion

Statement coverage alone cannot establish that nested loops terminate correctly. Basis-path testing, cyclomatic complexity, loop-boundary testing, and timeout-based checks provide stronger evidence that the migration process is reliable and preserves data integrity.

10. Food Delivery Application – State Transition Testing

Introduction

A mobile food delivery application tracks an order through the states Ordered, Preparing, Out for Delivery, and Delivered. Users report that the application becomes stuck when a driver cancels during delivery.

This indicates a missing or incorrectly implemented state transition.

Analysis of the Existing Black-Box Strategy

Equivalence Class Partitioning and Boundary Value Analysis are useful for validating individual inputs, but they do not fully represent the sequence of events in a dynamic workflow.

For example, a cancellation event may behave correctly before delivery but fail when it occurs after the driver has collected the order.

Application of State Transition Testing

State Transition Testing models the system's states, events, transitions, and expected results.

The normal workflow is:

Ordered → Preparing → Out for Delivery → Delivered.

The cancellation behavior must also be defined. Depending on the requirements, cancellation during delivery may trigger driver reassignment, return-to-store processing, a refund, or customer support.

The Delivered state should normally be terminal unless the business requirements explicitly permit another transition.

Proposed Test Scenarios

1. Ordered transitions successfully to Preparing.
2. Preparing transitions successfully to Out for Delivery.
3. Out for Delivery transitions successfully to Delivered.
4. Driver cancellation occurs before preparation.
5. Driver cancellation occurs during delivery.
6. A replacement driver is assigned.
7. A cancellation event is received more than once.
8. A delayed event arrives after the order has been delivered.
9. An invalid transition, such as Delivered back to Preparing, is attempted.
10. Network failure occurs during a state update.

For each scenario, verify the order status, customer notifications, payment or refund status, and consistency between the application and backend.

Improved Testing Strategy

Create a state-transition diagram and a transition table listing every permitted transition. Test all important valid transitions and selected invalid transitions, including cancellation and recovery scenarios.

Conclusion

State Transition Testing is especially suitable for dynamic workflows because it examines the relationship between the current state and incoming events. It helps identify stuck orders, invalid transitions, and inconsistent status updates that ECP and BVA alone may miss.

11. Cybersecurity Authentication Module – Hybrid Testing Using Error Guessing and Path Analysis

Introduction

A cybersecurity authentication module uses password strength, OTP timing, and IP-location checks. Despite testing, brute-force vulnerabilities were discovered.

This suggests that the testing process did not sufficiently examine repeated authentication failures, retry limits, timing rules, and combinations of authentication conditions.

Analysis of the Existing Testing Strategy

Black-box testing checks externally observable behavior, while white-box testing examines internal program logic.

If the team tested only ordinary successful and unsuccessful logins, it may have missed repeated attempts, OTP replay, account enumeration, or paths that bypass security checks.

Application of Error Guessing

Error guessing uses experience and knowledge of common defects to identify likely failure scenarios.

Test the following cases:

1. Repeated incorrect password attempts.
2. Repeated OTP submissions.
3. Expired OTP submission.
4. Reuse of an already accepted OTP.
5. Rapid authentication requests.
6. Concurrent login attempts.
7. Authentication from an unusual IP location.
8. Password-reset abuse.
9. Different error messages that reveal whether an account exists.
10. Attempts to bypass the rate limiter or multi-factor checks.

Verify that rate limiting, delays or lockout, monitoring, and account-recovery controls operate according to the security requirements.

Application of Control Flow and Path Analysis

Construct a CFG representing the authentication process:

Start → Rate-Limit Check → Password Verification → OTP Validation → IP/Risk Evaluation → Allow or Deny Access.

Identify independent paths for correct and incorrect passwords, valid and invalid OTPs, expired OTPs, trusted and risky IP locations, rate-limit violations, locked accounts, and service failures.

Verify that no execution path allows authentication without the required security checks.

Improved Hybrid Testing Strategy

Combine black-box negative testing and error guessing with white-box branch coverage and basis-path testing. Perform code reviews, authorized security testing, and automated regression tests for every identified vulnerability.

Coverage metrics alone do not prove security, so testing should also be guided by threat modeling and the system's security requirements.

Conclusion

Error guessing helps identify realistic attack scenarios, while path analysis examines whether internal control flow enforces authentication rules consistently. Their combination improves the system's resistance to brute-force attacks and strengthens overall security.

12. E-Learning Platform – Subscription Expiration and ECP

Introduction

An e-learning platform provides Free, Basic, and Premium subscriptions. Errors occur when a user upgrades on the exact day the subscription expires.

This suggests that the system may not correctly handle date boundaries or may have ambiguous rules regarding subscription validity.

Analysis of the Existing Testing Approach

The existing test cases may have checked dates before and after expiration but omitted the exact expiration date.

Another possible problem is that the application does not clearly define whether access remains valid through the expiration day or ends at the beginning of that day.

The expected behavior must be established from the subscription requirements.

Application of Equivalence Class Partitioning

Divide subscription dates into the following classes:

1. Date before subscription start.
2. Date within the valid subscription period.
3. Exact expiration date.
4. Date after expiration.
5. Invalid or missing subscription date.

Divide subscription types into Free, Basic, and Premium. Course types should also be classified as included, excluded, or restricted according to the subscription rules.

Boundary Value Testing

Test the day immediately before expiration, the exact expiration day, and the day immediately after expiration.

If the application uses timestamps, also test the final valid moment and the first invalid moment under the specified time-zone rules.

Proposed Test Scenarios

Test a Basic user upgrading to Premium before expiration, on the expiration date, and after expiration.

Also test a Free user attempting to access restricted content, a Premium user accessing an included course, an upgrade with invalid payment details, and an account with missing or invalid subscription dates.

Verify the subscription tier, effective dates, access permissions, payment status, and renewal behavior.

Improved Testing Strategy

Use ECP to select representative cases from each subscription and date class. Add explicit boundary tests and decision-table tests covering subscription type, course type, and expiration status.

Automate the tests and execute them after changes to subscription logic.

Conclusion

ECP provides systematic coverage of subscription and date classes, while boundary testing targets the exact point at which access rules change. Together, they help ensure correct subscription upgrades and seamless transitions between access levels.

13. Smart-Home Lighting Controller – Branch and Condition Coverage

Introduction

A smart-home lighting controller adjusts light intensity using ambient-light sensors, switch-case statements, and Boolean expressions.

Relying solely on statement coverage may leave defects undetected because the program may execute its statements without testing every decision outcome or Boolean condition.

Analysis of the Existing Approach

Statement coverage confirms that statements execute at least once, but it does not establish that all switch-case alternatives, Boolean conditions, and decision outcomes have been tested.

For example, a light-dimming statement may execute when ambient light is low, while the behavior for

adequate ambient light or an unoccupied room remains untested.

Application of Branch Coverage

Branch coverage requires each decision outcome to execute.

The team should test all relevant switch-case alternatives, including the default case, and both outcomes of each conditional statement.

Application of Condition Coverage

Condition coverage requires every individual Boolean condition to evaluate to both True and False at least once.

For example, consider the decision:

Dim the light when ambient light is low AND occupancy is detected.

Test the following combinations:

1. Low ambient light and occupancy detected.
2. Low ambient light and no occupancy.
3. Adequate ambient light and occupancy detected.
4. Adequate ambient light and no occupancy.

These cases help reveal errors in the decision logic.

Branch coverage and condition coverage are different. Condition coverage alone may not execute every decision outcome, while branch coverage alone may not demonstrate the independent effect of each condition.

Improved White-Box Testing Strategy

Construct a CFG representing the switch-case statements, Boolean decisions, sensor input handling, and hardware-control commands.

Test minimum and maximum sensor readings, disconnected sensors, unexpected switch values, abrupt ambient-light changes, and communication failures.

Use MC/DC for complex safety-relevant Boolean decisions when required. Test the controller using simulation or hardware-in-the-loop testing and verify both software commands and actual light responses.

Conclusion

Combining statement, branch, and condition coverage provides stronger evidence than statement coverage alone. CFG analysis and hardware integration tests further reduce the risk of failures caused by interactions between sensors, software logic, and lighting hardware.

14. Logistics Management System – Robust Boundary Value Analysis

Introduction

A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. The testing team failed to detect errors involving zero and negative weights.

This indicates that the existing input validation tests did not adequately cover invalid boundary values.

Analysis of the Existing Approach

Testing only valid positive weights does not establish whether the system rejects invalid values. If zero or negative weights are accepted, the application may calculate incorrect shipping charges or create invalid shipment records.

Application of Robust Boundary Value Analysis

Robust Boundary Value Analysis tests values at the limits and immediately outside the permitted range.

Assume the permitted weight range is from W_{min} to W_{max}, where $W_{min} > 0$.

Test the following values:

- $W_{min} - 1$ smallest supported unit.

- W_{min}.

- $W_{min} + 1$ smallest supported unit.

- A typical valid weight.
- $W_{max} - 1$ smallest supported unit.

- W_{max}.

- $W_{max} + 1$ smallest supported unit.

Also test zero, negative weight, blank input, non-numeric input, and excessive decimal precision.
For example, if the minimum permitted weight is 0.1 kg, test 0 kg, 0.1 kg, and 0.2 kg, along with negative values. The actual thresholds must come from the system requirements.
Additional Test Scenarios
Repeat selected weight-boundary cases across destination zones and delivery speeds to verify that invalid weights are rejected regardless of the selected shipping option.
Verify the calculated shipping charge, validation message, shipment-label creation, and prevention of invalid transactions.
Improved Testing Strategy
Combine robust BVA with ECP. ECP divides weights into valid and invalid classes, while BVA targets the exact limits where behavior changes.
Negative testing should be included in the regression suite to ensure that invalid inputs remain rejected after software changes.
Conclusion
Robust BVA detects defects immediately inside and outside the permitted weight range. Negative testing is necessary to ensure correct validation, prevent incorrect shipping charges, and improve the reliability of logistics operations.

15. Medical Imaging Software – Basis-Path Testing and Prime-Sized Datasets
Introduction
Medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur when the dataset size is a prime number.
This indicates a possible problem in data partitioning, remainder handling, synchronization, or parallel result aggregation.
Analysis of Random Testing
Random testing may discover unexpected defects, but it cannot guarantee that the specific prime-sized datasets associated with the failure will be tested.
A structured approach is necessary because prime numbers can cause uneven distribution of data among parallel workers.
Construction of the Control Flow Graph
Construct a CFG representing:
1. Dataset validation.
2. Dataset partitioning.
3. Assignment of data to parallel workers.

4. Parallel processing loops.
5. Handling of remaining elements.
6. Synchronization of workers.
7. Aggregation of results.
8. Error handling and program termination.

The graph should include branches for empty partitions, remainder elements, worker failures, and different dataset sizes.

Application of Basis-Path Testing

Calculate cyclomatic complexity using:

\[
V(G)=E-N+2P
\]

For a single connected graph:

\[
V(G)=E-N+2
\]

Identify independent paths and design test cases to execute each path. This helps cover internal control-flow alternatives that random testing may miss.

Proposed Structured Test Cases

Test datasets with:

- Size 0 and size 1, where permitted.
- Small prime sizes such as 2, 3, 5, and 7.
- Nearby composite sizes such as 4, 6, and 8.
- Sizes divisible evenly by the number of parallel workers.
- Sizes that leave remainder elements during partitioning.
- Large datasets near chunk-size boundaries.

Run the same input repeatedly under different worker counts and scheduling conditions to help identify race conditions.

Improved Testing Strategy

Compare parallel-processing results against a trusted sequential implementation or independently verified expected results.

Verify that every data element is processed exactly once, no elements are omitted or duplicated, and numerical results remain within the specified tolerance.

Combine basis-path testing with prime/composite input selection, loop-boundary testing, repeated execution, and appropriate thread-safety checks.

Conclusion

Random testing alone is insufficient for a defel accuracy, consistency, and system stability.